Aalto University
School of Science

Master's Programme in Computer, Communication and Information Sciences

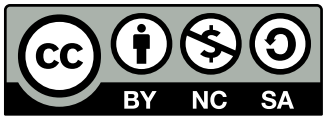
Context is king - Retrieval-Augmented Generation in enterprise-ready systems

Antti-Jussi Asunmaa

Master's Thesis
2026

© 2026

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 4.0 International" license.



Aalto University
School of Science

Author Antti-Jussi Asunmaa

Title Context is king - Retrieval-Augmented Generation in enterprise-ready systems

Degree programme Computer, Communication and Information Sciences

Major Computer Science

Supervisor Prof. Petri Vuorimaa

Advisor Lauri Salminen (Msc)

Collaborative partner Nordea Bank Oyj

Date 30 June 2025 **Number of pages** 49+2 **Language** English

Abstract

The abstract is a short description of the essential contents of the thesis, usually in one paragraph: what was studied and how and what were the main findings.

For a Finnish thesis, the abstract should be written in both Finnish and English; for a Swedish thesis, in Swedish and English. The abstracts for English theses written by Finnish or Swedish speakers should be written in English and either in Finnish or in Swedish, depending on the student's language of basic education. Students educated in languages other than Finnish or Swedish write the abstract only in English. Students may include a second or third abstract in their native language, if they wish.

The abstract text of this thesis is written on the readable abstract page as well as into the pdf file's metadata via the `\thesisabstract` macro (see comment in this TEX file above). Write here the text that goes onto the readable abstract page. You can have special characters, linebreaks, and paragraphs here. Otherwise, this abstract text must be identical to the metadata abstract text.

If your abstract does not contain special characters and it does not require paragraphs, you may take advantage of the `\abstracttext` macro (see the comment in this TEX file below).

Keywords

Aalto University
School of Science

Tekijä Antti-Jussi Asunmaa

Työn nimi Context is king - Retrieval-Augmented Generation in enterprise-ready systems

Koulutusohjelma Computer, Communication and Information Sciences

Pääaine Computer Science

Työn valvoja Prof. Petri Vuorimaa

Työn ohjaaja DI. Lauri Salminen

Yhteistyötaho Nordea Bank Oyj

Päivämäärä 30.6.2025 **Sivumäärä** 49+2 **Kieli** englanti

Tiivistelmä

Tiivistelmä on lyhyt kuvaus työn keskeisestä sisällöstä usein yhtenä kappaleena: mitä tutkittiin ja miten sekä mitkä olivat tärkeimmät tulokset. Suomenkielisen opinnäytteen tiivistelmä kirjoitetaan suomeksi ja englanniksi ja ruotsinkielisen vastaavasti ruotsiksi ja englanniksi. Suomen- tai ruotsinkielisten opiskelijoiden, joiden opinnäytteen kieli on englanti, tulee kirjoittaa tiivistelmänsä englanniksi ja koulusivistyskielellään. Muiden kuin koulusivistyskieleltään suomen- tai ruotsinkielisten tulee kirjoittaa tiivistelmänsä vain englanniksi. Opiskelija voi halutessaan lisätä opinnäytteeseensä toisen tai kolmannen tiivistelmän omalla äidinkielellään. Tämän opinnäytteen tiivistelmäteksti kirjoitetaan opinnäytteen luettavan osan lomakkeen lisäksi myös pdf-tiedoston metadataan. Kirjoita tähän metadataan kirjoitettavaa teksti. Metadatatekstissa ei saa olla erikoismerkkejä, rivinvaiho- tai kappaleenjakomerkkiä, joten näitä merkkeja ei saa käyttää tässä. Jos tiivistelmäsi ei sisällä erikoimerkkejä eikä kaipaa kappaleenjakoa, voit hyödynttää makroa abstracttext luodessasi lomakkeen tiivistelmää (katso kommentti tässä TeX-tiedostossa alla). Metadatatiivistelmatekstin on muuten oltava sama kuin lomakkeessa oleva teksti.

Avainsanat RAG

Preface

I want to thank Professor Pirjo Professor and my instructors Dr Alan Advisor and Ms Elsa Expert for their guidance.

I also want to thank my partner for keeping me sane and alive.

Otaniemi, 30 June 2025

Eddie E. Engineer

AI usage

Large language models (LLMs) and generative artificial intelligence tools, specifically Google Gemini, were utilized in multiple phases of this thesis. First, they assisted in the ideation phase to refine the research scope. Second, they were used to identify relevant literature, keywords, and search formulations for the literature review. Third, AI tools were employed for grammatical correction and to improve the stylistic flow and vocabulary of the thesis; however, all draft text was originally authored by the author. Lastly, AI-based code assistants were used to support code development for the empirical evaluation.

The author accepts full responsibility for the factual accuracy of this thesis and for any errors or omissions, whether originating from the author's own input or from the systems utilized.

Contents

Abstract 3

Abstract (in Finnish) 4

Preface 5

AI usage 6

Contents 7

List of Tables 9

Glossary & Abbreviations 10

1 Introduction 12

2 Technical background 14
2.1 Large-Language Models . . . 14
2.1.1 Evolution and Adoption of Language Models . . . 14
2.1.2 Architectural and Operational Constraints . . . 14
2.2 Naive Retrieval-Augmented Generation . . . 15

3 Literature review 18
3.1 Cascading . . . 18
3.2 Small-to-Big retrieval . . . 19
3.3 Semantic Re-ranking . . . 20
3.4 Prompt Caching . . . 22

4 System Architecture and Experimental Methodology 24
4.1 Research Methodology . . . 24
4.2 System Implementation and Models . . . 25
4.3 Optimization Pipeline . . . 26
4.4 Benchmark Datasets . . . 28
4.5 Evaluation Setup and Metrics . . . 28
4.5.1 Latency Profiles and Computational Overhead . . . 29
4.5.2 Operational Cost Accounting . . . 29
4.5.3 Quality Metrics . . . 30
4.5.4 Pareto Frontier and Trade-off Equilibrium . . . 32
4.6 Threats to Validity . . . 32

5 Results 34
5.1 Naive Baseline Performance and Workload Calibration . . . 34
5.2 Stage 1: Single-Factor Isolated Optimization . . . 34
5.2.1 Small-to-Big Retrieval (C_1): Context Coherence vs. Expansion Overhead . . . 34

5.2.2 Two-Stage Semantic Re-ranking (C_2): Distractor Filtering vs. Cross-Encoder Latency . . . 34
5.2.3 Dual-Tier Prompt Caching (C_3): Prefill Bypassing and Economic Scaling . . . 34
5.2.4 Hybrid Model Cascading (C_4): Workload-Aware Complexity Tiering . . . 34
5.3 Stage 2: Combinatorial Pipeline Scaling and Multi-Module Interactions 34
5.3.1 Pairwise Synergies and Structural Conflicts ($C_5 - C_{10}$) . . . 34
5.3.2 Multi-Module Triplets and Full-Stack Integration ($C_{11} - C_{15}$) 34
5.3.3 Stacking Anomalies and the Law of Diminishing Returns . . 35
5.4 Multi-Objective Synthesis and Trade-off Equilibrium . . . 35
5.4.1 Master Benchmark Performance and Statistical Testing . . . 35
5.4.2 Latency Profiles and Computational Overhead (RQ1) . . . 35
5.4.3 Operational Cost Scaling and Token Economics (RQ2) . . . 35
5.4.4 Multidimensional Response Quality Dynamics (RQ3) . . . 35
5.4.5 Pareto Frontier and Enterprise Deployment Archetypes (RQ4) 35
5.4.6 Summary Resolution of Research Questions . . . 35

6 Discussion 42

7 Conclusion 43

A Enterprise Knowledge Corpus Extraction Methodology 50

B Operational Cost Catalog and Cloud Tariffs 51

List of Tables

1 Primary research questions 13
2 Full factorial experimental design matrix 25
3 Two-tailed hypothesis formulations mapped to the research questions (C_k vs. C_0). 26
4 Architectural specifications of optimization modules 27
5 Benchmark evaluation performance stratified by MuSiQue reasoning depth tiers. 34
6 Quantitative performance, operational cost, and response quality across RAG pipeline configurations. 35
7 Paired Wilcoxon signed-rank test results and Holm-Bonferroni step-down corrections (Thesis Table 4.2). 36
8 Summary resolution of thesis research questions mapped to empirical findings. 41
B1 Operational pricing catalog and cloud tariffs for generative, re-ranking, and embedding services (verified as of September 2026). 51

Glossary & Abbreviations

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
LitM	Lost in the Middle
LLM	Large Language Model
NLP	Natural Language Processing
PLM	Pre-trained Language Model
RAG	Retrieval-Augmented Generation
TTFT	Time-to-First-Token

Glossary

Chunking	The process of dividing a large source text into smaller, distinct segments (chunks) prior to vector database insertion.
Context Window	The maximum amount of text (measured in tokens) that a language model can process in a single prompt iteration.
Cosine Similarity	A mathematical metric used to measure the semantic similarity between two vectors by calculating the cosine of the angle between them.
Embedding	A mathematical vector representation of text that captures its semantic and contextual meaning in a high-dimensional space.
Hallucination	A phenomenon where a language model generates output that is fluent and plausible but factually incorrect or unsupported by the training data.
Inference	The operational phase where a fully trained language model processes a user prompt and generates an output response.
Non-parametric Knowledge	Data or information that resides outside the language model's internal weights, typically stored in an external database, and retrieved dynamically during inference.
Parametric Knowledge	The static, internal information that is permanently embedded within a language model's weights during its training phase.
Time-to-First-Token	A performance metric that measures the latency from the moment a prompt is sent to the model until it begins generating its very first output token.
Token	The foundational atomic unit of text (such as a word, character, or sub-word) into which input strings are split before being processed by a language model.
Vectorization	The algorithmic process of converting textual data into numerical vectors (embeddings) using a machine learning model.

1 Introduction

Modern enterprise-grade AI systems face continuously increasing resource constraints, in addition to growing operational expenditures driven by a rapid acceleration in corporate AI integration and exponential growth in the demand for AI computing capacity [1]. Although API providers frequently lower unit token pricing, overall costs continue to rise due to explosive token usage and the complexity of multi-step agentic workflows that use the latest foundational large language models (LLMs) [2, 3]. This highlights the critical need for optimization across all phases of enterprise AI systems to ensure that computing resources are utilized as efficiently as possible.

This is especially true for Retrieval-Augmented Generation (RAG), which serves as a primary solution to mitigate critical limitations of modern LLMs, such as hallucination, the lack of up-to-date data, and the absence of domain-specific knowledge [4, 5]. By design, RAG inherently increases token consumption, particularly input tokens, as the core concept relies on prepending retrieved document context to user queries to generate more grounded responses. Within the retrieval module, a central challenge lies in finding the optimal balance between retrieval efficiency and the quality of the retrieved information [5]. While various optimization strategies have been proposed in contemporary literature [5, 6], they are typically evaluated on static benchmarks and lack direct integration with industry-standard, production-grade languages and frameworks such as Java and Spring AI [7].

The primary objective of this thesis is to investigate state-of-the-art optimization strategies for RAG systems, and the main contribution is to evaluate how these strategies perform within enterprise environments. Furthermore, this work proposes a concrete architectural solution to implement these optimization strategies utilizing enterprise frameworks. To establish a focused scope, four specific optimization strategies are analyzed in detail: model cascading [2], small-to-big retrieval [8], semantic re-ranking [9], and prompt caching [10].

Each of these methods addresses a distinct phase of the RAG pipeline. Model cascading and small-to-big retrieval optimize the query and retrieval stages, whereas semantic re-ranking refines the retrieved results to minimize prompt size and maximize contextual relevance. Prompt caching, on the other hand, serves to store previous responses in a cache so that subsequent, similar queries can be resolved immediately, thereby bypassing the computationally intensive retrieval and generation pipeline.

Many of these optimization strategies are typically evaluated in isolation, leaving a gap in the literature regarding how they interact when combined. For example, combining semantic re-ranking and small-to-big retrieval could increase the risk of cache misses in prompt caching, thereby reducing its effectiveness. In addition, while model cascading may increase *per-query* latency for complex tasks routed to more capable models, it decreases *average* latency overall by resolving simpler tasks with faster, less capable models.

Evaluating and resolving these interactive conflicts is crucial for high-volume enterprise

applications, such as customer support systems, where operational requirements are exceptionally rigorous. In these environments, minor delays and incremental token usage quickly compound into systemic instability, elevated latencies, and escalating operational costs. Furthermore, hallucinated or erroneous outputs present substantial risks of reputational damage and potential financial liabilities [11]. Because enterprise systems must strictly navigate these critical constraints, cost, latency, and output quality are established as the primary evaluation metrics for this thesis. To guide this investigation, this study addresses the four primary research questions summarized in Table 1.

Table 1: Primary research questions addressed in this thesis.

	Research Question
RQ1	**Latency:** How do small-to-big retrieval, semantic re-ranking, prompt caching, and model cascading affect the latency profile (specifically Time to First Token [TTFT] and overall end-to-end execution latency) of a RAG system?
RQ2	**Cost:** To what extent do these optimization methods reduce the operational cost of the system, accounting for token pricing tiers, prompt caching discounts, and model cascading?
RQ3	**Quality:** How do these optimizations impact retrieval and generation quality (measured by Context Precision, Context Recall, Faithfulness, and Answer Relevance) compared to a naive RAG baseline?
RQ4	**Interaction:** How do these optimization methods interact when combined, and what is the optimal equilibrium between latency, cost, and quality in an enterprise customer support system?

The rest of the thesis is structured as follows: Section 2 briefly explains what LLMs and RAG are, while Section 3 examines the current literature on RAG optimization methods. Research methods are introduced in Section 4, and the results are discussed in Section 5. Finally, the discussion is presented in Section 6, and concluding remarks are provided in Section 7.

2 Technical background

RAG offers an architectural solution to mitigate several of the most fundamental limitations of LLMs, such as hallucination, the lack of up-to-date data, and the lack of domain-specific knowledge [5]. To establish a comprehensive understanding of these concepts, this chapter provides the necessary technical background for both technologies. The first section examines the architectural foundations and inherent drawbacks of LLMs[1], while the subsequent section focuses on the core principles and operational mechanics of plain vanilla RAG.

2.1 Large-Language Models

2.1.1 Evolution and Adoption of Language Models

Natural language processing (NLP) has traditionally focused on enabling computer systems to understand human-generated language and acquire knowledge [13]. Early statistical approaches, such as n-gram language models, attempted to resolve the inherent ambiguity of natural language by predicting the probability distribution of word sequences [13]. While these traditional models could perform structured tasks like rule-based information extraction or spam filtering, their capability to comprehend semantic nuances and manage complex linguistic tasks remained limited [14].

Many pre-trained language models (PLMs), such as GPT-2 [15], are based on the transformer architecture [16], which excels at handling long-range dependencies through parallel training. While early PLMs required task-specific fine-tuning to perform downstream applications [17], modern LLMs scale this architecture to hundreds of billions of parameters. According to Zhao et al. [14], the transition from PLMs to LLMs is primarily characterized by this massive scale in parameters and training data, which gives rise to emergent abilities, enabling the models to solve complex tasks and follow instructions without explicit task-specific retraining.

Presently, LLMs are among the most widely used technologies behind modern generative AI applications [14]. Even though these models have demonstrated strong performance across various tasks, such as coding [18], and have been widely adopted[2], they are not perfect.

2.1.2 Architectural and Operational Constraints

Despite their capabilities, LLMs exhibit several major limitations, including biases, hallucinations, and the "Lost in the Middle" (LitM) phenomenon [19, 20]. Bias refers to the fact that the model's output heavily depends on its training data, meaning that the generated content may include inappropriate language, conspiracy theories, or gender bias [19].

[1] This section is partly based on the author's bachelor's thesis [12].

[2] For instance, data aggregated in the report [1] highlights a rapid acceleration in corporate AI integration. In Europe, corporate adoption in at least one business function was 91% in 2025 compared to 57% in 2023.

Due to the autoregressive nature of modern LLMs, these models are inherently prone to hallucinations, where they produce outputs that appear plausible but are factually incorrect [21]. This occurs because the models lack an intrinsic understanding of facts, but instead, they generate responses by predicting subsequent tokens based on statistical patterns learned during training. This relying mechanism draws entirely from the model's parametric knowledge, meaning the information is permanently embedded within the model's weights during the training phase [17]. This static nature, consequently, makes it computationally and environmentally expensive and impractical to frequently retrain or fine-tune entire models when the underlying information required to complete a task changes [1].

While the context windows, i.e., LLMs' input sizes, have grown significantly during the last couple of years, allowing models to process up to millions of input tokens [22, 23], these architectures consistently suffer from the LitM phenomenon. This problem refers to a limitation where models effectively utilize information located at the absolute beginning (primacy bias) or the end (recency bias) of the prompt, while failing to retrieve relevant facts buried within the middle sections [20]. In addition, utilizing massive contexts severely increases computational latency and operational expenses, as modern cloud LLM APIs employ token-based pricing models where input costs scale linearly with prompt length and often double when exceeding specific context thresholds [3]. These inherent limitations effectively prevent the full utilization of large context windows in production-grade enterprise environments when attempting to ingest massive enterprise knowledge bases directly into the prompt.

Another source of errors in the output is that the training data has a cutoff date, so it does not contain the most recent information and may generate incorrect answers to questions about recent events [24]. To mitigate these limitations, modern LLM-based systems are integrated with web search capabilities to retrieve real-time information from the internet, a paradigm used, for example, by the Google Gemini model family [22]. While this integration helps keep responses up to date, it introduces an external dependency on the reliability of internet sources. Furthermore, relying purely on open-web search does not solve the challenge for enterprise environments, where the required knowledge is locked behind internal intranet pages and private corporate databases rather than publicly accessible web pages.

2.2 Naive Retrieval-Augmented Generation

To address the limitations detailed in the previous section, Retrieval-Augmented Generation (RAG) offers an architectural solution that decouples the LLM's reasoning capabilities from its static parametrized memory [4]. RAG enriches the LLMs' parametric knowledge with non-parametric data served dynamically from an external knowledge base. The LLM's output is based on the user's initial prompt, augmented with the data retrieved from the external knowledge base, and therefore forces the LLM to ground its response in the provided context rather than its parametrized data.

The retrieval-augmentation process requires both the source knowledge and the user

query to be preprocessed. Preprocessing of the knowledge base contains three phases: chunking, vectorizing, and indexing. During this data preparation phase, source documents — such as PDF files — are chunked into smaller, more manageable segments[3]. These chunks are converted into numerical vectors using an embedding model so that their semantic meanings can be mathematically calculated, and they are subsequently stored in an indexed vector database [26]. During runtime inference, the user's initial prompt is vectorized using the same embedding model. The system then calculates the mathematical distance (e.g., cosine similarity) between the prompt vector and the stored vectors, retrieving a pre-defined number of the most relevant chunks — denoted as the top-k — from the database to augment the initial prompt [27].

To formalize this two-phase pipeline, the retrieval-augmentation process can be represented mathematically. In the first phase, a document corpus is split into N smaller text chunks, forming the external knowledge base $D = \{d_1, d_2, \ldots, d_N\}$. These chunks are then mapped into a vector database using an embedding function $E(\cdot)$, which yields the vector representation $\mathbf{v}_i = E(d_i)$ for each chunk $d_i \in D$.

In the second phase, the user's query q is embedded into a vector form using the same embedding function $E(q)$, subsequently, after which the top k most relevant chunks R_q are fetched based on their cosine similarity score using the retrieval function Ret_k:

$$\mathrm{Ret}_k(q, D) = R_q. \tag{1}$$

Finally, the query q and the retrieved chunks R_q are fed to the generator model $G(\cdot)$ to produce the final answer a, denoted as

$$G(q, R_q) = a. \tag{2}$$

The primary advantage of the RAG architecture lies in its reliance on an external knowledge base. When properly prompted, this mechanism significantly mitigates hallucinations by forcing the LLM to ground its output exclusively on the retrieved context [28]. Furthermore, maintaining an external data repository is substantially more cost-effective and computationally practical than continuously retraining or fine-tuning the underlying model [26], allowing enterprise systems to remain up-to-date in real time. From a data governance perspective, an external knowledge base ensures that sensitive corporate data remains secure and under internal control [29]. Lastly, by selectively retrieving only the most relevant documents, RAG compresses the required input context, thereby reducing API transaction costs and inference latency compared to naive long-context processing.

However, these benefits introduce distinct trade-offs. The major drawback of a naive RAG pipeline is the increased end-to-end latency of the system [26]. This latency overhead stems primarily from sequential network requests: first querying the

[3]Early architectures like Lewis et al. [4] utilized fixed-length segments of 100 words, while modern architectures optimize segment lengths [25].

embedding model and vector database, and subsequently transmitting the expanded prompt to the LLM. While modern vector distance calculations are highly optimized, the network overhead and the increased time-to-first-token (TTFT) caused by processing larger, augmented prompts add noticeable delay [26]. Consequently, if the RAG pipeline is not properly optimized, the final prompt can grow excessively large, rendering the production system both financially expensive and operationally slow.

3 Literature review

This chapter reviews the current literature on the four selected optimization techniques, focusing on the performance benefits each method provides in relation to the research questions. Relevant publications were identified on Google Scholar using keyword combinations of the optimization method and "RAG" or "retrieval-augmented generation". In addition, snowballing [30], starting from comprehensive surveys (e.g., [31, 26, 5]), was used to discover further relevant studies. Due to the rapid pace of development in this research area, the selected literature consists primarily of articles published after 2023. The remainder of this chapter is structured as follows: Section 3.1 reviews model cascading and routing, Section 3.2 examines small-to-big retrieval, Section 3.3 covers semantic re-ranking, and Section 3.4 discusses prompt caching.

3.1 Cascading

Modern RAG systems encounter diverse workloads, ranging from simple informational queries to highly complex analytical tasks. In production-grade enterprise environments — where high availability, low latency, and cost efficiency are critical constraints — a "one-size-fits-all" architecture is fundamentally sub-optimal [32, 2]. It over-allocates expensive computational resources for trivial questions, yet may lack the retrieval depth and reasoning capacity required for complex queries. To mitigate this, model cascading or query routing can be employed to dynamically direct inputs to an appropriately scaled configuration of the RAG pipeline [33]. For instance, a simple query can bypass the full pipeline and utilize a lightweight LLM with minimal context. Conversely, a complex query requiring synthesis across multiple enterprise knowledge bases can trigger an augmented pipeline with extended retrieval top-k parameters and a more capable foundation model.

To optimize these trade-offs systematically, these dynamic architectures can be formalized mathematically. In a classification-based query routing system, the selection process is modeled by defining a set of available generator models, $\mathcal{G} = \{G_1, G_2, \ldots, G_m\}$, ordered by their capacity and cost. Based on the estimated complexity of the input query q, a routing classifier:

$$f_{\text{route}}(q) = j \in \{1, \ldots, m\}. \tag{3}$$

maps it to a specific model index. Consequently, the query q and the retrieved chunks R_q are fed to the selected generator model $G_j(\cdot)$ to produce the final answer a:

$$G_j(q, R_q) = a. \tag{4}$$

Model cascading offers substantial operational cost reductions both in isolated LLM invocations and in RAG pipelines whose overall cost profiles are dominated by API usage. For instance, FrugalGPT [2] routes individual requests sequentially from lightweight to heavyweight models based on a reliability score generated by a scoring classifier. This cascading approach achieves up to a 98.3% cost reduction compared

to a homogeneous GPT-4 baseline while maintaining comparable response quality. Similarly, RouteLLM [33] yields a 1.14× to 3.66× cost reduction compared to a static GPT-4 deployment by dynamically bypassing expensive models, preserving approximately 95% of the baseline's quality. Similar cost reduction patterns have also been demonstrated by other adaptive RAG frameworks [32, 34].

Routing low-complexity queries to smaller models yields significant latency benefits due to their faster inference speeds. Crucially, the computational overhead introduced by the routing decision itself remains minimal, typically amounting to approximately 11 ms per query [35] with an added routing cost overhead below 0.4% [33]. In straightforward scenarios, bypassing retrieval operations entirely can yield order-of-magnitude improvements: Jeong et al. [32] reports a reduction in generation latency from 3.08 and 27.18 seconds (for single-step and multi-step retrieval pipelines) to 0.35 seconds. Furthermore, adaptive stopping mechanisms, such as that proposed by Park et al. [36], can prune unnecessary retrieval-generation loops, reducing the total number of iterations by 14–49%.

While providing both operational cost and latency benefits, query routing has a modest impact on quality. For example, Jeong et al. [32] found that the F1 score remained practically unchanged, while achieving a substantial reduction in latency (56%) and in token consumption (63%) compared to a multi-step retrieval baseline (using GPT-3.5). Similar results have been shown by Guo et al. [34], where token consumption decreased by up to 90% while losing only 0.8 points on F1, but improving the Exact Match score by 2.6 points compared to the agentic baseline, and yielding 20.7 and 15.5 percentage point improvements in F1 and Exact Match scores, respectively, compared to naive RAG.

3.2 Small-to-Big retrieval

Document chunking is a foundational phase in RAG pipelines, directly determining the granularity of the vector index. However, selecting a static chunk size introduces a fundamental trade-off between retrieval precision and contextual completeness. While smaller chunks are computationally efficient and exhibit high intrachunk cohesion (ICC), they frequently suffer from low document contextual coherence (DCC) due to semantic fragmentation [25]. Conversely, larger chunks preserve rich contextual relationships but degrade retrieval accuracy by diluting query-relevant semantic features within dense vector representations. Small-to-big retrieval resolves this tension by decoupling the representation used for vector search from the context provided to the generator. In this architecture, dense retrieval is executed over fine-grained index nodes, while the generation phase utilizes a broader, mapped context (such as parent chunks or entire source documents) retrieved via sibling pointer mapping. Implementations of this technique range from neighboring chunk expansion, which dynamically retrieves adjacent sliding windows to restore structural continuity [8], to hierarchical abstract trees, which map granular leaf nodes to high-level thematic summaries[37].

To abstract these various approaches, the underlying retrieval process is formally

modeled by representing the search space as a set of fine-grained index nodes $D_{\text{small}} = \{d_1, \ldots, d_N\}$ and a context-mapping function $g : D_{\text{small}} \to D_{\text{parent}}$ that maps each small chunk to its corresponding parent chunk or expanded context window. The retriever first operates on the small chunks:

$$\text{Ret}_k(q, D_{\text{small}}) = R_q^{\text{small}}. \tag{5}$$

Finally, the query q and the mapped parent contexts are fed to the generator model $G(\cdot)$ to produce the final answer a:

$$G\left(q, \{g(d) \mid d \in R_q^{\text{small}}\}\right) = a. \tag{6}$$

Different chunking and representation methods directly increase the accuracy of RAG pipelines. For example, Lumer et al. [8] showed that expanding the context to neighboring chunks can increase the win rate to 65% compared to using single independent chunks. In addition, the adaptive chunking method, where chunks are generated dynamically based on document structure, increases downstream answer correctness by 8–10 percentage points without requiring downstream prompt modifications [25]. Hierarchical abstract trees like Bridge-RAG [37] yield up to a 44% improvement in BLEU scores on complex reasoning tasks by tracing parent-child lineages. Similarly, dynamic reasoning frameworks like LogicRAG [38] and QPaug [39] increased string accuracy by 14.7% absolute (compared to the next best graph baseline) and answer F1 scores by over 34% relatively.

While small-to-big strategies provide substantial benefits to answer correctness and retrieval recall, they introduce operational and computational overhead. At query time, asynchronous parallel search for neighboring chunks [8] keeps latency minimal, adding a negligible ≈ 0.17 s overhead. However, more complex reasoning structures like LogicRAG [38] more than double the query latency compared to naive flat retrieval, increasing it from 4.28 s to 9.83 s. Furthermore, advanced chunking and indexing methods are highly time-consuming during the ingestion phase. The slowest model-driven and layout-aware chunking methods can run for tens of hours, whereas fixed-size chunking completes in under a second [40]. In hierarchical structures, generating node summaries and index trees also increases offline preprocessing costs (e.g., scaling from \$1.19 to \$6.12 per average filing in financial domains) [8]. Nonetheless, this preprocessing latency is typically acceptable in enterprise environments, as chunking and indexing are offline processes that occur only during initial document ingestion.

3.3 Semantic Re-ranking

In RAG pipelines, retrieved chunks might exhibit high mathematical similarity (e.g., cosine similarity) but lack actual semantic relevance to the query. The worst-case result of this semantic discrepancy is incorrect generation, compounded by the unnecessary token overhead of feeding irrelevant data to the generator. Semantic re-ranking is an optimization method that employs a multi-stage retrieval architecture to address

this issue. First, a fast and computationally efficient bi-encoder (such as dense vector embeddings or sparse BM25 retrieval) is used to retrieve a large candidate set (e.g., top-100) to optimize the recall phase. Subsequently, a computationally intensive cross-encoder reranking model evaluates deep query-document interactions to refine the results and select the top-k most relevant chunks before the generation of the final answer [41, 42]. Another key benefit of semantic re-ranking is its ability to mitigate the LitM phenomenon [20] by sorting the most semantically relevant chunks to the top of the context window. Furthermore, by filtering out irrelevant chunks before the generation stage, the reranker compresses the final prompt size, which directly reduces both input token consumption and TTFT.

To model this process systematically, semantic re-ranking is formalized as a two-stage retrieval pipeline. Let $D = \{d_1, d_2, \ldots, d_N\}$ represent the database of document chunks, and let q denote the user query. In the first stage, a computationally efficient bi-encoder retriever selects a large candidate set $C_1 \subset D$ of size K_1 (where $K_1 \gg k$):

$$R_{K_1}(q, D) = C_1. \tag{7}$$

In the second stage, a cross-encoder model reranks the candidates in C_1 based on deep query-document interactions to select the top-k most relevant chunks, forming the pruned context $C_2 \subset C_1$:

$$\text{Rerank}_k(q, C_1) = C_2. \tag{8}$$

Finally, the query q and the pruned context C_2 are fed to the generator model $G(\cdot)$ to produce the final answer a:

$$G(q, C_2) = a. \tag{9}$$

Semantic re-ranking has been shown to significantly improve both the retrieval and generation quality of RAG pipelines. For instance, in open-domain question-answering, Moreira et al. [41] demonstrated that a second-stage neural reranker yields a 14% relative and a 9 percentage point absolute improvement in average NDCG@10 across the BEIR QA datasets compared to standard bi-encoder retrievers. This improvement translates directly to downstream accuracy: by filtering out non-contributing documents, Wang et al. [43] increased Exact Match (EM) scores by 17.8 percentage points on NaturalQA and 17.4 percentage points on PopQA compared to naive RAG, while outperforming alternative self-reflective and corrective frameworks on TriviaQA with a 76.2% EM score. Furthermore, rather than relying on separate pipeline components, Yu et al. [9] consolidated ranking and generation into a single multi-task model, achieving 50.6% EM on NaturalQA, 82.9% EM on TriviaQA, and 57.6% EM on PopQA—outperforming standard chat-adapted baselines by 5.0 to 8.2 percentage points. When models are optimized for explicit denoising using self-synthesized rationales, they achieve relative accuracy improvements of 8.3% and demonstrate robust performance as the candidate count scales [44]. Finally, in domain-specific applications where precision is safety-critical, hybrid retrieval paired with a cross-encoder reranker increases high-correctness answers from 33.5% to 49.0% while reducing severe hallucinations and completely incorrect outputs from 35.3% to 22.5% [42].

Increased quality in semantic re-ranking comes with the explicit cost of increased latency while providing implicit potential for token savings. The key overhead arises during the second-stage retrieval, where the computationally intensive reranking model remains the primary bottleneck. For example, Moreira et al. [41] reports that using a pruned 4-billion-parameter reranker adds approximately 266 ms of latency on average. Similarly, incorporating the ranking mechanism directly into the generator model, as done in RankRAG, introduces a 0.9× to 6.0× latency overhead when evaluating between 20 and 100 candidate documents [9]. While downstream token savings are rarely quantified empirically in these studies, overall token consumption is implicitly reduced either by dynamically filtering out irrelevant documents below a relevance threshold [43] or by scaling down the final context window size since the most relevant documents are prioritized at the top [9].

3.4 Prompt Caching

RAG pipelines incur substantial TTFT latency and computational overhead because queries are continuously augmented with extensive retrieved context. However, much of this content is static or repetitive—such as system instructions, templates, and recurring document chunks. Prompt caching avoids redundant prefill passes by persisting intermediate key-value (KV) attention states in memory for reuse across requests. Modern architectures primarily use two paradigms: model-level prefix KV caching and application-level semantic completion caching. Prefix KV caching stores attention tensors for static prefixes or document blocks to bypass redundant prefill computation. Conversely, semantic completion caching stores historical query–response pairs, directly returning pre-computed answers when a query exceeds a semantic similarity threshold.

To formalize context-level KV caching systematically, the constructed prompt $P(q, R_q)$ is partitioned into a static prefix P_{static} and a dynamic, query-specific suffix $P_{\text{dynamic}}(q)$:

$$P(q, R_q) = [P_{\text{static}}, P_{\text{dynamic}}(q)]. \tag{10}$$

Once P_{static} has been pre-computed, its key-value attention states $KV(P_{\text{static}})$ are retrieved directly from the cache, allowing the generator model $G(\cdot)$ to evaluate self-attention exclusively over the dynamic suffix and produce the final answer:

$$G(P_{\text{dynamic}}(q) \mid KV(P_{\text{static}})) = a. \tag{11}$$

Prompt caching yields substantial latency reductions and operational cost savings across diverse RAG architectures. From a commercial deployment perspective, major cloud API providers offer prompt caching discounts up to a 10× cost reduction for cached prompt tokens [3]. At the infrastructure level, Gim et al. [10] reported 5×–10× TTFT speedups on VRAM-cached GPUs (1.5×–3× on host DRAM)—slashing latency from 900 ms to 90 ms on Llama-7B with 3K tokens—and up to 70× on CPUs. Similarly, Jin et al. [45] achieved a 4.0× TTFT reduction over vLLM+Faiss alongside a 2.1× throughput speedup via multi-tier knowledge tree caching. Broader evaluations

in TurboRAG [46], CacheBlend [47], and FusionRAG [48] confirm speedups ranging from 2.2× (CacheBlend) to 9.39× (FusionRAG). Although linear memory scaling presents a key trade-off (e.g., 2.5 GB RAM per 1K tokens for Llama-70B), tiered architectures mitigate VRAM constraints by offloading inactive KV blocks to DRAM or NVMe SSDs. Furthermore, by recomputing only critical tokens ($\leq$ 15% in FusionRAG and 15% in CacheBlend), these frameworks eliminate up to 82.3% of redundant prefill computation while sustaining cache hit rates as high as 77.9%.

Beyond latency and cost reductions, prompt caching maintains generation quality comparable to full prefill passes, with architectures like RAGCache [45] demonstrating zero quality loss across standard benchmarks. The primary challenge of prompt caching lies in uncaptured cross-chunk semantics and positional displacements, where identical context blocks exhibit varying semantics or positional shifts across queries, leading to cache misses and potential accuracy degradation. To overcome these limitations, recent approaches introduce selective recomputation strategies. For example, CacheBlend [47] selectively recomputes cross-attention for the top 15% most sensitive tokens (from a candidate set of 10–20% high-deviation tokens), restoring quality to within 0.01–0.03 absolute F1/ROUGE-L score of full prefill. Similarly, FusionRAG [48] pairs offline similarity-guided preprocessing with online query-guided attention selection—recomputing under 15% of tokens—to recover approximately 99% of the full attention quality.

4 System Architecture and Experimental Methodology

This chapter establishes the experimental framework and software architecture used to empirically evaluate the performance, operational cost, and output quality of RAG optimization techniques. To address the research questions formulated in Section 1, this study implements a quantitative benchmarking suite built upon an enterprise-ready Java and Spring AI infrastructure connected to Google Cloud Platform (GCP) services. The remainder of this chapter is structured as follows: Section 4.1 details the overall research approach and experimental design, while Section 4.2 describes the software architecture and cloud infrastructure. Section 4.3 outlines the specific optimization strategies evaluated. Section 4.4 introduces the target benchmark datasets, Section 4.5 specifies the quantitative evaluation metrics and measurement instrumentation, and Section 4.6 analyzes potential threats to experimental validity.

4.1 Research Methodology

The empirical research is conducted using quantitative benchmarking following a two-stage experimental strategy. In the first stage, each optimization module is evaluated in isolation against the naive RAG baseline (C_0) to quantify its individual performance characteristics. In the second stage, combinatorial configurations—spanning pairwise pairings, multi-module triplets, and the fully integrated full-stack pipeline—are systematically tested against the baseline to identify the Pareto frontier and investigate systemic trade-offs between combined techniques. The complete experimental design matrix is presented in Table 2. This evaluation framework directly addresses the research questions formulated in Table 1. To ensure statistical reliability, eliminate cold-start distortions (such as JVM Just-In-Time compilation, database connection pool initialization, and TLS handshakes), and mitigate transient network jitter, two unmeasured warm-up queries precede the evaluation of the benchmark suite. Each configuration in the $2^4 = 16$ experimental matrix is evaluated across the 100 benchmark queries per dataset (yielding a sample size of $N = 100$ paired observations per dataset, or $N = 200$ across both corpora), resulting in 3,200 total measured pipeline invocations across the experimental matrix.

Due to the non-deterministic characteristics of distributed cloud systems and network environments, the benchmarked latency and cost metrics are expected to exhibit non-normal distributions. Consequently, the non-parametric Wilcoxon signed-rank test is employed to evaluate the statistical significance of observed differences between each optimization configuration ($C_1, \ldots, C_{15}$) and the naive baseline (C_0). Because each pipeline configuration is evaluated across an identical, paired set of benchmark queries, the paired signed-rank formulation is strictly applicable.

Tests are conducted independently across each evaluation metric using a two-tailed formulation to detect both performance enhancements and potential degradation without directional bias. To mitigate the inflated risk of Type I errors resulting from

Table 2: Full factorial experimental design matrix ($2^4 = 16$ configurations) across the two-stage benchmarking framework.

Config	Pipeline Composition	M_1: S2B	M_2: SR	M_3: PC	M_4: MC
Baseline ($k = 0$)					
C_0	Naive RAG Baseline	–	–	–	–
Stage 1: Single-Factor Isolated Testing ($k = 1$)					
C_1	S2B	✓	–	–	–
C_2	SR	–	✓	–	–
C_3	PC	–	–	✓	–
C_4	MC	–	–	–	✓
Stage 2: Pairwise Combinations ($k = 2$)					
C_5	S2B + SR	✓	✓	–	–
C_6	S2B + PC	✓	–	✓	–
C_7	S2B + MC	✓	–	–	✓
C_8	SR + PC	–	✓	✓	–
C_9	SR + MC	–	✓	–	✓
C_{10}	PC + MC	–	–	✓	✓
Stage 2: Multi-Module Triplets ($k = 3$)					
C_{11}	S2B + SR + PC	✓	✓	✓	–
C_{12}	S2B + SR + MC	✓	✓	–	✓
C_{13}	S2B + PC + MC	✓	–	✓	✓
C_{14}	SR + PC + MC	–	✓	✓	✓
Stage 2: Full-Stack Integration ($k = 4$)					
C_{15}	S2B + SR + PC + MC (Full-Stack)	✓	✓	✓	✓

Notes: M_1: Small-to-Big Retrieval (S2B); M_2: Semantic Re-ranking (SR); M_3: Prompt Caching (PC); M_4: Model Cascading (MC). Checkmarks (✓) denote active optimization components in the pipeline.

multiple pairwise comparisons (15 configurations across multiple metrics), family-wise error rates are controlled by applying the Holm-Bonferroni correction to all raw p-values. Statistical significance is evaluated against a threshold of $\alpha = 0.05$. Formal hypothesis definitions mapped to the research questions are presented in Table 3. Note that RQ4 examines multi-objective trade-offs and is evaluated using Pareto frontier analysis (Section 4.5.4) rather than a statistical hypothesis test.

4.2 System Implementation and Models

The experimental testbed is built upon a production-grade enterprise software stack. The application is implemented in Java 21 LTS using Spring Boot 4.1.0 and Spring AI version 2.0.0. Relational metadata and vector embeddings are managed in PostgreSQL version 16 with the `pgvector` extension (version 0.8.0), utilizing Hierarchical Naviga-

Table 3: Two-tailed hypothesis formulations mapped to the research questions (C_k vs. C_0).

Target	Metric	Null Hypothesis (H_0)	Alternative Hypothesis (H_1)
RQ1	Latency (TTFT, E2E)	$\tilde{\mu}_{\Delta\text{Latency}} = 0$	$\tilde{\mu}_{\Delta\text{Latency}} \neq 0$
RQ2	Cost (USD / tokens)	$\tilde{\mu}_{\Delta\text{Cost}} = 0$	$\tilde{\mu}_{\Delta\text{Cost}} \neq 0$
RQ3	Quality (RAGAS)	$\tilde{\mu}_{\Delta\text{Quality}} = 0$)	$\tilde{\mu}_{\Delta\text{Quality}} \neq 0$

Notes: $\tilde{\mu}_{\Delta}$ represents the median paired difference between configuration C_k and the naive baseline C_0. Tests are evaluated at significance level $\alpha = 0.05$.

ble Small World (HNSW) indexing with cosine distance [4]. The index uses standard parameters ($M = 16$, $ef_construction = 64$, $ef_search = 40$) to achieve high retrieval recall with sub-5 ms search latency. The software stack is deployed on Google Cloud Platform (GCP), with the database hosted on fully managed Google Cloud SQL instances to replicate a realistic enterprise production environment. Services are co-located within a dedicated Virtual Private Cloud (VPC) using private IP peering in the `europe-north1` region to minimize round-trip time (RTT) and eliminate network latency variance.

The primary foundation model for generation is Google Gemini 3.8 Flash; for model cascading (M_4), Gemini 3.5 Flash-Lite and Gemini 3.1 Pro are utilized as low-latency and high-capacity tiers, respectively. Dense vector representations are generated using `gemini-embedding-2`, configured with an output dimensionality of 768 via Matryoshka Representation Learning (MRL) [49] to balance retrieval efficiency with storage requirements. For two-stage semantic re-ranking (M_2), Google Cloud's Vertex AI Ranking API with the `semantic-ranker-default-004` cross-encoder model is employed.

4.3 Optimization Pipeline

Each optimization method encompasses a broad architectural design space. For example, chunk token limits and overlap ratios vary substantially across RAG literature. Furthermore, differences in foundation model parameter scales, routing thresholds, and caching configurations significantly expand the potential parameter space. To ensure rigorous and reproducible evaluation, the technical implementation and baseline configurations evaluated in this study are summarized in Table 4. The selected parameters balance empirical prevalence in the literature review (Section 3) with practical feasibility in enterprise software stacks.

Naive Baseline Configuration (M_0). A fixed chunk size of 512 tokens with a 50-token overlap and dense vector retrieval of top-$k = 5$ is chosen as the baseline configuration, reflecting standard experimental benchmarks in recent literature [40, 4].

Table 4: Architectural specifications and technical implementation of the evaluated RAG optimization modules.

Module	Optimization Strategy	Variables and Configuration
M_0	Naive RAG (Baseline)	Chunk size: 512 tokens, overlap: 50 tokens, retrieval: top-k = 5.
M_1	Small-to-Big Retrieval	Child chunk: 128 tokens, parent window: 1024 tokens, retrieval: top-k = 5.
M_2	Two-Stage Semantic Re-ranking	Candidate pool: top-50, reranked context: top-k = 5, model: `semantic-ranker-default-004`.
M_3	Dual-Tier Prompt Caching	Context threshold: $\geq$ 4096 tokens, caching mechanism: GCP Explicit Context Cache / KV prefix reuse.
M_4	Hybrid Model Routing	Routing targets: Router/1-hop $\rightarrow$ Gemini 3.5 Flash-Lite; 2-hop $\rightarrow$ Gemini 3.8 Flash; 3/4-hop $\rightarrow$ Gemini 3.1 Pro.

Small-to-Big Retrieval (M_1). Child chunks of 128 tokens expanding into 1024-token parent context windows are selected based on empirical configurations in recent hierarchical RAG studies [25, 37, 8]. Furthermore, when multiple retrieved child chunks originate from the same parent document, a deduplication and interval-merging step collapses overlapping boundaries into a single parent context window to prevent redundant token consumption and context duplication.

Two-Stage Semantic Re-ranking (M_2). The candidate retrieval pool for M_2 is generated using dense vector embeddings from `gemini-embedding-2` and re-ranked using the Google Cloud Vertex AI Ranking API with the `semantic-ranker-default-004` cross-encoder model. The candidate pool size depends on the active pipeline composition: in isolated configurations (C_2), first-stage vector search retrieves a candidate pool of 50 chunks of 512 tokens each to match the baseline granularity. In combined configurations ($C_5, C_{11}, C_{12}, C_{15}$), the first-stage vector search retrieves 50 child chunks of 128 tokens each, which are re-ranked directly by `semantic-ranker-default-004` before the top-5 winning chunks are expanded to their 1024-token parent context windows.

Dual-Tier Prompt Caching (M_3). A minimum threshold of 4096 tokens is selected to satisfy the operational boundary of Google Cloud Vertex AI Context Caching for static system instructions and pre-indexed knowledge corpora. In addition, dynamically retrieved chunks and conversational turns benefit from automatic KV cache prefix reuse across sequential interactions.

Hybrid Model Routing (M_4). To replicate real-world enterprise deployments where query complexity is unknown a priori, an automated zero-shot classifier powered by Gemini 3.5 Flash-Lite dynamically evaluates incoming queries. Straightforward single-hop requests are handled directly by Gemini 3.5 Flash-Lite, intermediate queries requiring multi-passage synthesis are escalated to Gemini 3.8 Flash, and complex multi-hop reasoning tasks are routed to Gemini 3.1 Pro. The latency overhead (T_{route}) and token costs of the classifier are explicitly recorded in the evaluation.

4.4 Benchmark Datasets

The primary academic benchmark dataset is MuSiQue [50], from which a sample of 100 question–answer pairs is selected following established benchmarking methodologies in recent RAG literature [8, 25]. Specifically, the sample comprises 40% 2-hop questions and 60% 3- and 4-hop questions to ensure balanced coverage across varying reasoning difficulty tiers. Multi-hop queries eliminate the reasoning shortcuts and surface-level lexical matching common in single-hop datasets [50], making them well-suited for evaluating Small-to-Big retrieval (M_1), Semantic Re-ranking (M_2), and Prompt Caching (M_3). Furthermore, stratifying the evaluation set by reasoning depth provides a realistic testbed for assessing hybrid Model Cascading (M_4) across distinct query complexity tiers.

To complement the academic benchmark with domain-specific enterprise workloads, a secondary evaluation dataset comprising 100 synthetically generated question–answer pairs is constructed to reflect real-world customer inquiries regarding the products and services of the partner enterprise. The underlying knowledge corpus is compiled by scraping the public-facing documentation and web portal of the partner organization, with the comprehensive extraction methodology detailed in Appendix A. To evaluate pipeline robustness across varied interaction patterns, the dataset is partitioned into 40% single-hop factual queries, 40% multi-hop relational questions, and 20% out-of-domain queries. For out-of-domain queries, the expected reference response is a standardized refusal string (*"The provided context does not contain sufficient information to answer this query."*), allowing rejection accuracy and hallucination resistance to be quantitatively evaluated.

4.5 Evaluation Setup and Metrics

The empirical evaluation of system performance, operational expenditure, and response quality is conducted using the industry-standard RAGAS evaluation suite [51] combined with enterprise-grade application telemetry. Within the RAGAS framework, Claude Sonnet 5 is deployed as an automated LLM-as-a-judge at zero temperature ($T = 0.0$) to ensure deterministic evaluation scoring, following established standards in the literature. The following subsections formally establish the measurement instrumentation and mathematical formulations for each research question.

4.5.1 Latency Profiles and Computational Overhead

The latency characteristics of the RAG pipelines are evaluated across two primary operational dimensions: Time to First Token (TTFT, in milliseconds) and overall End-to-End latency (T_{total}, in milliseconds). TTFT measures the duration elapsed from query submission until the generation model emits its initial response token, serving as a critical indicator of perceived user responsiveness in interactive systems. End-to-end latency quantifies the total wall-clock execution time required for the entire pipeline to complete generation and stream the response. To isolate the computational overhead introduced by individual architectural optimizations, fine-grained telemetry records the execution latency of each micro-phase:

$$\begin{aligned} T_{\text{total}}(q) = T_{\text{embed}}(q) + T_{\text{route}}(q) + T_{\text{ret}}(q) + T_{\text{rerank}}(q) \\ + T_{\text{expand}}(q) + T_{\text{cache}}(q) + T_{\text{prefill}}(q) + T_{\text{stream}}(q) \end{aligned} \tag{12}$$

where T_{embed} denotes query vectorization duration, T_{route} is the model cascading routing overhead, T_{ret} is the first-stage HNSW vector search latency, T_{rerank} is the cross-encoder semantic re-ranking duration, T_{expand} is the parent-document metadata resolution time, T_{cache} is the context cache lookup and validation latency, T_{prefill} is the generator prefill duration, and T_{stream} is the streaming token generation duration. For pipeline configurations where specific optimization modules are inactive, their corresponding latency terms evaluate to zero. Under this formulation, user-perceived Time to First Token is defined as:

$$\text{TTFT}(q) = T_{\text{total}}(q) - T_{\text{stream}}(q) \tag{13}$$

Both median (P_{50}) and 95th percentile tail (P_{95}) latencies are reported across the steady-state executions to evaluate enterprise SLA compliance.

4.5.2 Operational Cost Accounting

Operational expenditure constitutes a critical bottleneck for enterprise RAG adoption at scale. To establish an accurate, reproducible economic model, execution costs are computed per query based on published Google Cloud Gemini Enterprise Agent Platform pricing tiers. The operational cost model incorporates dynamic model tier routing (M_4), context caching discounts (M_3), and auxiliary service invocation overhead (M_2). The total monetary cost $\text{Cost}(q)$ for processing query q is formally defined as:

$$\begin{aligned} \text{Cost}(q) = N_{\text{embed}}(q) \cdot P_{\text{embed}} + N_{\text{uncached}}(q) \cdot P_{\text{prompt}}^{(m(q))} + N_{\text{cached}}(q) \cdot P_{\text{cached}}^{(m(q))} \\ + N_{\text{comp}}(q) \cdot P_{\text{comp}}^{(m(q))} + N_{\text{rerank}}(q) \cdot P_{\text{rerank}} \end{aligned} \tag{14}$$

where $m(q) \in \{\text{Flash-Lite}, \text{Flash}, \text{Pro}\}$ represents the active generative model tier selected for query q, N_{embed} is the number of query tokens vectorized at unit rate P_{embed}, N_{uncached} and N_{cached} represent uncached and cached prompt tokens billed at tier-specific rates $P^{(m(q))}_{\text{prompt}}$ and $P^{(m(q))}_{\text{cached}}$ (reflecting the 90% discount applied to cached input context), N_{comp} denotes the generated completion tokens billed at rate $P^{(m(q))}_{\text{comp}}$, and N_{rerank} represents the count of candidate passages evaluated by the cross-encoder at per-document rate P_{rerank}. Fixed persistent infrastructure expenses, such as base Cloud SQL hosting, are invariant across configurations and therefore excluded from per-query operational cost calculations. The complete unit tariff schedule, encompassing all evaluated generative model tiers, cross-encoder ranking services, and embedding models alongside official vendor documentation references and acquisition timestamps (September 2026), is detailed in Appendix B (Table B1).

Because individual execution expenditures in highly optimized or cached architectures are fractional (frequently falling below $0.001 per request), presenting single-query costs directly introduces significant rounding truncation in empirical summaries. Consequently, operational expenditures in benchmark summary tables and Pareto frontier analyses are normalized and reported per 1,000 queries:

$$\text{Cost}_{1\text{k}} = 1000 \times \frac{1}{|Q|} \sum_{q \in Q} \text{Cost}(q) \tag{15}$$

providing an intuitive, standard enterprise pricing metric that transparently reflects macroscopic operational volume while preserving underlying per-query accounting fidelity.

4.5.3 Quality Metrics

Latency reduction and operational cost minimization are viable only if the response quality of the RAG pipeline satisfies end-user requirements. RAG pipelines exhibit two primary failure modes: retrieval failure and generation failure. Retrieval failure is defined as the inability to retrieve relevant context passages or the inclusion of irrelevant noise, whereas generation failure occurs when, despite retrieving relevant context, the generator produces an inaccurate response or hallucinated claims.

To evaluate quality across both stages of the pipeline, the RAGAS [51] framework provides dedicated metrics, formulated in Equations 16–19. While RAGAS is subject to the stochastic nature of LLM evaluation judges (discussed in Section 4.6), objective, deterministic token-level lexical overlap metrics—specifically token F_1 and Exact Match (EM)—are additionally computed against gold standard reference answers. Finally, a composite quality score is calculated to synthesize overall pipeline performance for the Pareto trade-off analysis in Section 4.5.4.

The evaluated RAGAS quality metrics are defined as follows:

Context Precision. Context Precision measures the retriever's ability to rank relevant chunks higher than irrelevant ones in the retrieved context [51]:

$$\text{Context Precision@}K = \begin{cases} \frac{\sum_{k=1}^{K}(\text{Precision@}k \times v_k)}{\sum_{k=1}^{K} v_k}, & \text{if } \sum_{k=1}^{K} v_k > 0 \\ 0, & \text{otherwise} \end{cases} \tag{16}$$

where K is the total number of retrieved context chunks, $v_k \in \{0, 1\}$ represents the binary relevance indicator at rank k, and $\text{Precision@}k = \frac{\text{true positives@}k}{\text{true positives@}k + \text{false positives@}k}$.

Context Recall. Context Recall evaluates whether the retrieved context covers all necessary reference statements required to answer the query [51]:

$$\text{Context Recall} = \frac{|\text{Ground Truth Sentences Attributed to Context}|}{|\text{Total Sentences in Ground Truth}|} \tag{17}$$

In edge cases where the reference ground truth contains no factual statements (e.g., in valid out-of-domain queries), Context Recall defaults to 1.0 to prevent division by zero.

Faithfulness. Faithfulness measures the factual consistency of the generated response against retrieved context R_q, penalizing ungrounded hallucinations [51]:

$$\text{Faithfulness} = \frac{|\text{Claims in Answer Inferred from Context}|}{|\text{Total Claims in Generated Answer}|} \tag{18}$$

For valid refusals where the generated answer contains no affirmative factual claims, Faithfulness defaults to 1.0.

Answer Relevancy. Answer Relevancy quantifies how directly the generated response addresses the user's initial prompt, penalizing incomplete answers or extraneous verbosity [51]:

$$\text{Answer Relevancy} = \frac{1}{N} \sum_{i=1}^{N} \frac{E(q) \cdot E(q_i^{\text{gen}})}{\|E(q)\| \|E(q_i^{\text{gen}})\|} \tag{19}$$

where N denotes the number of synthetic questions generated from the model's response by the evaluation judge (default $N = 3$), q_i^{gen} is the i-th generated question, and $E(\cdot)$ represents the dense embedding function used to compute cosine similarity against original query q.

Composite Quality Score. The Composite Quality Score $Q(q) \in [0, 1]$ synthesizes the four individual quality dimensions from Equations 16–19 as a regularized harmonic mean:

$$Q(q) = \frac{4}{\frac{1}{\mathrm{CP}(q)+\epsilon} + \frac{1}{\mathrm{CR}(q)+\epsilon} + \frac{1}{\mathrm{Faith}(q)+\epsilon} + \frac{1}{\mathrm{AR}(q)+\epsilon}} \tag{20}$$

where $\mathrm{CP}(q)$, $\mathrm{CR}(q)$, $\mathrm{Faith}(q)$, and $\mathrm{AR}(q)$ denote Context Precision, Context Recall, Faithfulness, and Answer Relevancy for query q, respectively, and $\epsilon = 10^{-4}$ is a minimal smoothing parameter introduced to guarantee numerical stability and penalize catastrophic failures in any single quality dimension.

4.5.4 Pareto Frontier and Trade-off Equilibrium

To identify the Pareto frontier and quantify the trade-offs between response quality, operational expenditure, and execution speed, two practical efficiency ratios are formulated: Quality per Dollar (QPD) and Quality per Second (QPS). These composite indicators capture the return on investment (ROI) for each optimization technique and multi-module combination across the experimental matrix:

$$\mathrm{QPD}(q) = \frac{Q(q)}{\mathrm{Cost}(q)} \tag{21}$$

$$\mathrm{QPS}(q) = \frac{Q(q)}{T_{\mathrm{total}}(q)/1000} \tag{22}$$

where $Q(q)$ is the composite quality score from Equation 20, $\mathrm{Cost}(q)$ is the total monetary execution cost in USD from Equation 14, and $T_{\mathrm{total}}(q)/1000$ represents end-to-end execution latency converted from milliseconds to seconds from Equation 12. Together, these ratios allow systematic identification of configurations that maximize generative quality while minimizing operational latency and cost overhead.

4.6 Threats to Validity

According to Wohlin et al. [52], empirical research in software engineering is subject to three primary categories of threats to validity: construct validity, internal validity, and external validity. Construct validity concerns whether the experimental instrumentation accurately measures the target theoretical concepts. Internal validity addresses whether observed performance differences are strictly attributable to the evaluated optimization methods rather than confounding system noise. Finally, external validity evaluates the degree to which the experimental findings generalize beyond the specific evaluation testbed.

Construct Validity. The RAGAS evaluation framework relies heavily on an LLM-as-a-judge. This dependency and the stochastic nature of LLMs introduce potential threats to validity, as the judge's evaluations may exhibit non-deterministic behavior. In addition, if the evaluation judge and the generative pipeline shared the same model family, the results would be susceptible to vendor self-preference bias. To mitigate these risks, an independent model from a different family, Claude Sonnet 5, is selected as the judge, configured with a temperature of $T = 0.0$ to ensure deterministic evaluation scoring. Furthermore, to complement the model-based evaluation, deterministic token-level lexical overlap metrics (F_1 and Exact Match) are computed against gold standard reference answers.

Internal Validity. Distributed computer systems and network environments introduce numerous sources of latency variance, ranging from cold-start overheads to transient network jitter. To mitigate cold-start distortions (such as JVM Just-In-Time compilation, database connection pool initialization, and TLS handshakes), multiple unmeasured warm-up runs are executed prior to recording steady-state measurements. Additionally, the software stack and database are deployed within a dedicated VPC in the `europe-north1` region to minimize network hops and isolate the testbed from public internet routing fluctuations. Finally, application and database caches are explicitly reset between distinct experimental configurations to prevent cross-run state leakage.

External Validity. Practically all of the systems introduced in Section 3 are implemented using a very homogeneous set of programming languages, relying mostly on Python. A primary motivation of this study is to examine how these optimization concepts generalize into enterprise-standard languages and frameworks, specifically Java and Spring AI. There is also a risk that the pre-training corpus of the underlying LLMs might include academic benchmarks used in this study, potentially enabling the model to answer queries based on memorized parametric knowledge rather than retrieved context. To mitigate this issue, the standardized MuSiQue dataset [50] is paired with a domain-specific synthetic customer support dataset to assess model generalizability.

5 Results

5.1 Naive Baseline Performance and Workload Calibration

Table 5: Benchmark evaluation performance stratified by MuSiQue reasoning depth tiers.

Reasoning Tier	N	P_{50} (ms)	P_{95} (ms)	Tokens avg	Cost ($) per 1k q	EM (%)	F1 (%)	Faith	AR	CP	C
2-Hop Reasoning	5	638.0	1470.0	4888	$0.741	0.0%	3.2%	0.400	0.165	0.400	
Combined Corpus	5	638.0	1470.0	4888	$0.741	0.0%	3.2%	0.400	0.165	0.400	

Notes: Evaluated over the MuSiQue multi-hop benchmark sample partitioned across 2-hop, 3-hop, and 4-hop questions. Costs reflect Google Cloud Gemini Enterprise platform pricing normalized per 1,000 queries. Metrics follow Chapter 4 definitions.

5.2 Stage 1: Single-Factor Isolated Optimization

5.2.1 Small-to-Big Retrieval (C_1): Context Coherence vs. Expansion Overhead

5.2.2 Two-Stage Semantic Re-ranking (C_2): Distractor Filtering vs. Cross-Encoder Latency

5.2.3 Dual-Tier Prompt Caching (C_3): Prefill Bypassing and Economic Scaling

5.2.4 Hybrid Model Cascading (C_4): Workload-Aware Complexity Tiering

5.3 Stage 2: Combinatorial Pipeline Scaling and Multi-Module Interactions

5.3.1 Pairwise Synergies and Structural Conflicts ($C_5 - C_{10}$)

The Retrieval Synergy (C_5: Small-to-Big + Re-ranking).

The Caching Friction (C_8: Re-ranking + Prompt Caching).

The Maximum Efficiency Pair (C_{10}: Prompt Caching + Model Cascading).

5.3.2 Multi-Module Triplets and Full-Stack Integration ($C_{11} - C_{15}$)

Multi-Module Triplets ($C_{11} - C_{14}$).

Full-Stack Integration (C_{15}).

5.3.3 Stacking Anomalies and the Law of Diminishing Returns

5.4 Multi-Objective Synthesis and Trade-off Equilibrium

5.4.1 Master Benchmark Performance and Statistical Testing

Table 6: Quantitative performance, operational cost, and response quality across RAG pipeline configurations.

Configuration	N	P_{50} (ms)	P_{95} (ms)	Tokens avg	Cost ($) per 1k q	EM (%)	F1 (%)	Faith	AR	CP	CR
C0_baseline	5	638.0	1470.0	4888	$0.741	0.0%	3.2%	0.400	0.165	0.400	0.400
C11_M1_M2_M3	5	3375.0	4347.2	4790	$0.233	0.0%	9.5%	0.322	0.953	0.567	0.400
C1_M1	5	1137.0	2873.6	5120	$0.774	0.0%	0.0%	0.600	0.000	0.200	0.400
C2_M2	5	798.0	1239.4	4892	$0.741	0.0%	3.2%	0.800	0.105	0.573	0.400
C3_M3	5	3353.0	3615.0	4670	$0.228	0.0%	11.2%	0.333	0.906	0.500	0.400
C5_M1_M2	5	994.0	9945.2	5007	$0.759	0.0%	9.4%	0.600	0.387	0.567	0.400
C6_M1_M3	5	2491.0	3417.4	4881	$0.222	0.0%	6.8%	0.293	0.972	0.200	0.400
C8_M2_M3	5	3537.0	4964.2	4671	$0.227	0.0%	15.2%	0.264	0.704	0.573	0.400

Notes: P_{50} and P_{95} represent median and 95th percentile end-to-end execution latencies. Cost reflects Google Cloud Gemini Enterprise platform pricing normalized per 1,000 queries. EM: Exact Match; F1: Token-level F_1; Faith: Faithfulness; AR: Answer Relevancy; CP: Context Precision; CR: Context Recall; Comp Q: Harmonic mean Composite Quality score (Eq. 238).

5.4.2 Latency Profiles and Computational Overhead (RQ1)

5.4.3 Operational Cost Scaling and Token Economics (RQ2)

5.4.4 Multidimensional Response Quality Dynamics (RQ3)

5.4.5 Pareto Frontier and Enterprise Deployment Archetypes (RQ4)

5.4.6 Summary Resolution of Research Questions

Table 7: Paired Wilcoxon signed-rank test results and Holm-Bonferroni step-down corrections (Thesis Table 4.2).

RQ	Config	Evaluation Metric	C_0 Med	C_k Med	$\tilde{\mu}_\Delta$	W-Stat	p_{raw}	p_a
RQ1	C11_M1_M2_M3	End-to-End Latency (ms)	638.0	3375.0	+2737.0	0.0	0.0625	1.000
RQ2	C11_M1_M2_M3	Operational Cost ($)	0.0007	0.0002	-0.0005	0.0	0.0625	1.000
RQ3	C11_M1_M2_M3	Exact Match (EM)	0.0000	0.0000	+0.0000	0.0	1.0000	1.000
RQ3	C11_M1_M2_M3	Token F1 Score	0.0000	0.0845	+0.0845	3.5	0.3750	1.000
RQ3	C11_M1_M2_M3	Faithfulness	0.0000	0.1111	+0.0000	3.0	0.6250	1.000
RQ3	C11_M1_M2_M3	Answer Relevancy	0.0000	0.9583	+0.9402	0.0	0.0625	1.000
RQ3	C11_M1_M2_M3	Context Precision	0.5000	0.8333	+0.0000	0.0	0.5000	1.000
RQ3	C11_M1_M2_M3	Context Recall	0.0000	0.0000	+0.0000	0.0	1.0000	1.000
RQ3	C11_M1_M2_M3	Composite Quality Q	0.0000	0.0000	+0.0000	5.0	0.6250	1.000
RQ1	C1_M1	End-to-End Latency (ms)	638.0	1137.0	+522.0	4.0	0.4375	1.000
RQ2	C1_M1	Operational Cost ($)	0.0007	0.0008	+0.0000	0.0	0.0625	1.000
RQ3	C1_M1	Exact Match (EM)	0.0000	0.0000	+0.0000	0.0	1.0000	1.000
RQ3	C1_M1	Token F1 Score	0.0000	0.0000	+0.0000	0.0	1.0000	1.000
RQ3	C1_M1	Faithfulness	0.0000	1.0000	+0.0000	2.0	1.0000	1.000
RQ3	C1_M1	Answer Relevancy	0.0000	0.0000	+0.0000	0.0	1.0000	1.000
RQ3	C1_M1	Context Precision	0.5000	0.0000	+0.0000	0.0	1.0000	1.000
RQ3	C1_M1	Context Recall	0.0000	0.0000	+0.0000	0.0	1.0000	1.000
RQ3	C1_M1	Composite Quality Q	0.0000	0.0000	+0.0000	3.0	1.0000	1.000
RQ1	C2_M2	End-to-End Latency (ms)	638.0	798.0	+149.0	5.0	0.6250	1.000
RQ2	C2_M2	Operational Cost ($)	0.0007	0.0007	+0.0000	5.0	0.6250	1.000
RQ3	C2_M2	Exact Match (EM)	0.0000	0.0000	+0.0000	0.0	1.0000	1.000
RQ3	C2_M2	Token F1 Score	0.0000	0.0000	+0.0000	0.0	1.0000	1.000
RQ3	C2_M2	Faithfulness	0.0000	1.0000	+0.0000	0.0	0.5000	1.000
RQ3	C2_M2	Answer Relevancy	0.0000	0.0000	+0.0000	0.0	1.0000	1.000
RQ3	C2_M2	Context Precision	0.5000	0.8667	+0.0000	0.0	0.5000	1.000
RQ3	C2_M2	Context Recall	0.0000	0.0000	+0.0000	0.0	1.0000	1.000
RQ3	C2_M2	Composite Quality Q	0.0000	0.0000	+0.0000	0.0	0.1250	1.000
RQ1	C3_M3	End-to-End Latency (ms)	638.0	3353.0	+2487.0	0.0	0.0625	1.000
RQ2	C3_M3	Operational Cost ($)	0.0007	0.0002	-0.0005	0.0	0.0625	1.000
RQ3	C3_M3	Exact Match (EM)	0.0000	0.0000	+0.0000	0.0	1.0000	1.000
RQ3	C3_M3	Token F1 Score	0.0000	0.1449	+0.1449	2.0	0.1875	1.000
RQ3	C3_M3	Faithfulness	0.0000	0.5000	+0.0000	4.5	1.0000	1.000
RQ3	C3_M3	Answer Relevancy	0.0000	0.8852	+0.8852	0.0	0.0625	1.000
RQ3	C3_M3	Context Precision	0.5000	0.5000	+0.0000	0.0	1.0000	1.000
RQ3	C3_M3	Context Recall	0.0000	0.0000	+0.0000	0.0	1.0000	1.000
RQ3	C3_M3	Composite Quality Q	0.0000	0.0000	+0.0000	5.0	0.6250	1.000
RQ1	C5_M1_M2	End-to-End Latency (ms)	638.0	994.0	+344.0	0.0	0.0625	1.000
RQ2	C5_M1_M2	Operational Cost ($)	0.0007	0.0007	+0.0000	4.0	0.4375	1.000
RQ3	C5_M1_M2	Exact Match (EM)	0.0000	0.0000	+0.0000	0.0	1.0000	1.000
RQ3	C5_M1_M2	Token F1 Score	0.0000	0.0000	+0.0000	0.0	0.5000	1.000
RQ3	C5_M1_M2	Faithfulness	0.0000	0.5000	+0.0000	1.5	0.7500	1.000
RQ3	C5_M1_M2	Answer Relevancy	0.0000	0.0000	+0.0000	0.0	0.5000	1.000
RQ3	C5_M1_M2	Context Precision	0.5000	0.8333	+0.0000	0.0	0.5000	1.000
RQ3	C5_M1_M2	Context Recall	0.0000	0.0000	+0.0000	0.0	1.0000	1.000

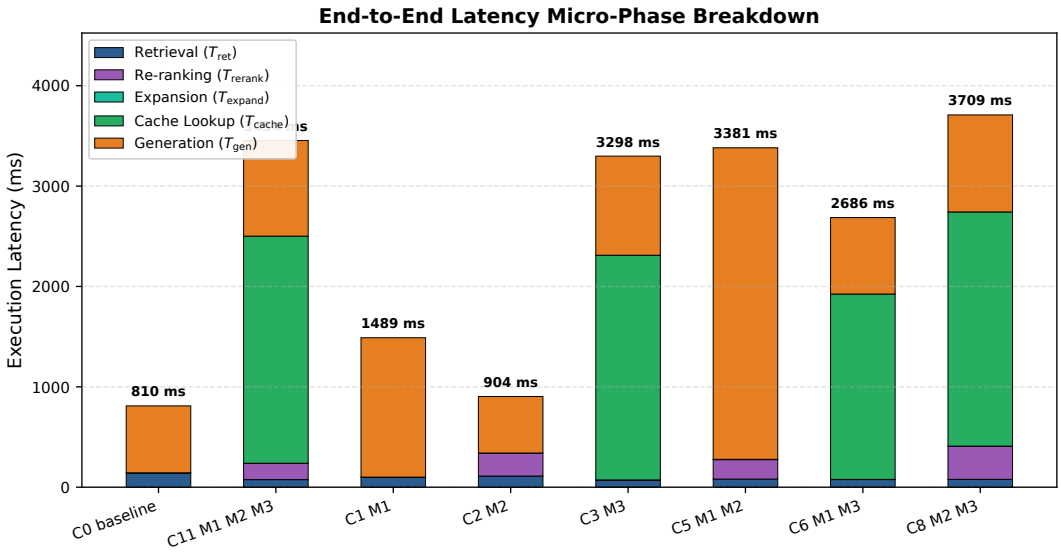


Figure 1: End-to-end latency micro-phase breakdown isolating vector retrieval (T_{ret}), semantic re-ranking (T_{rerank}), context expansion (T_{expand}), and generation (T_{gen}) durations across pipeline configurations. Execution latencies reflect steady-state observations.

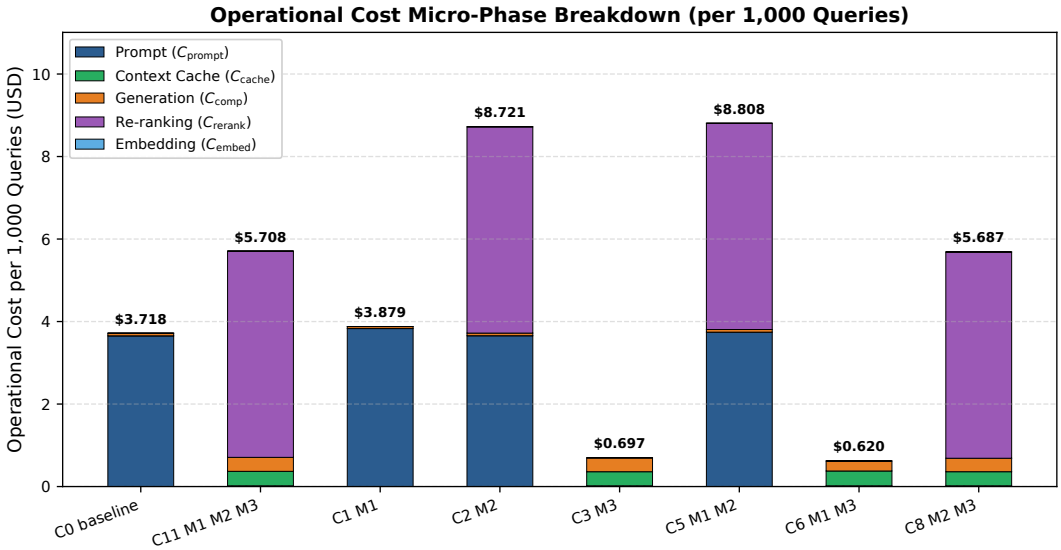


Figure 2: Operational cost micro-phase breakdown per 1,000 queries isolating uncached prompt tokens (C_{prompt}), context caching read tokens (C_{cache}), and generative completion tokens (C_{comp}) across pipeline configurations.

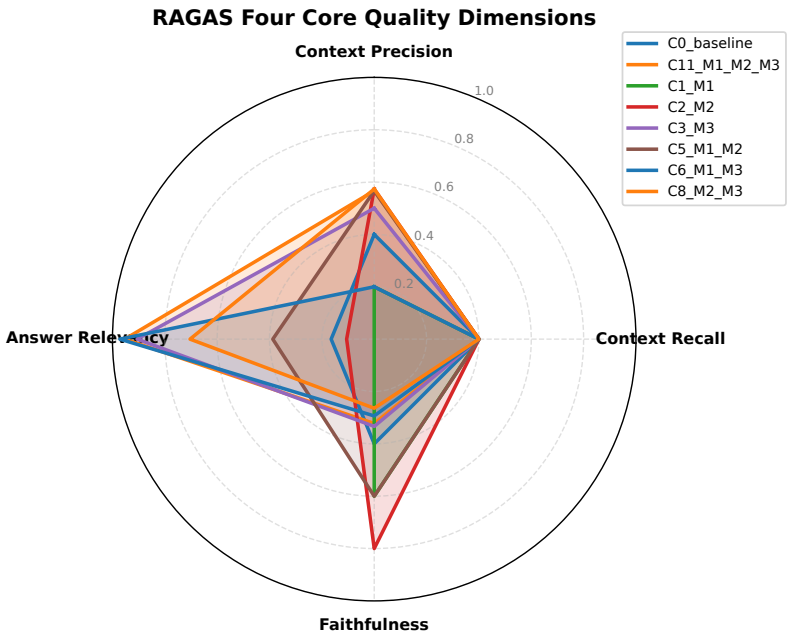


Figure 3: Multidimensional comparison of RAGAS quality metrics (Context Precision, Context Recall, Faithfulness, Answer Relevancy) across key pipeline configurations **[PLACEHOLDER VISUALIZATION]**.

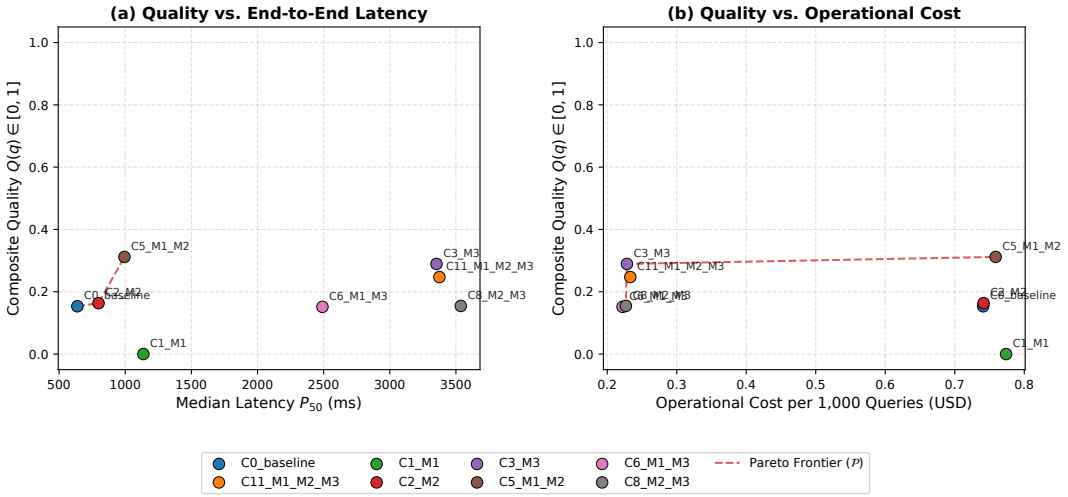


Figure 4: Empirical Pareto frontiers across RAG pipeline configurations: (a) Composite Quality vs. Median Latency (P_{50}), and (b) Composite Quality vs. Operational Cost per 1,000 Queries **[PLACEHOLDER VISUALIZATION]**. Dashed red lines denote the non-dominated Pareto frontier ($\mathcal{P}$).

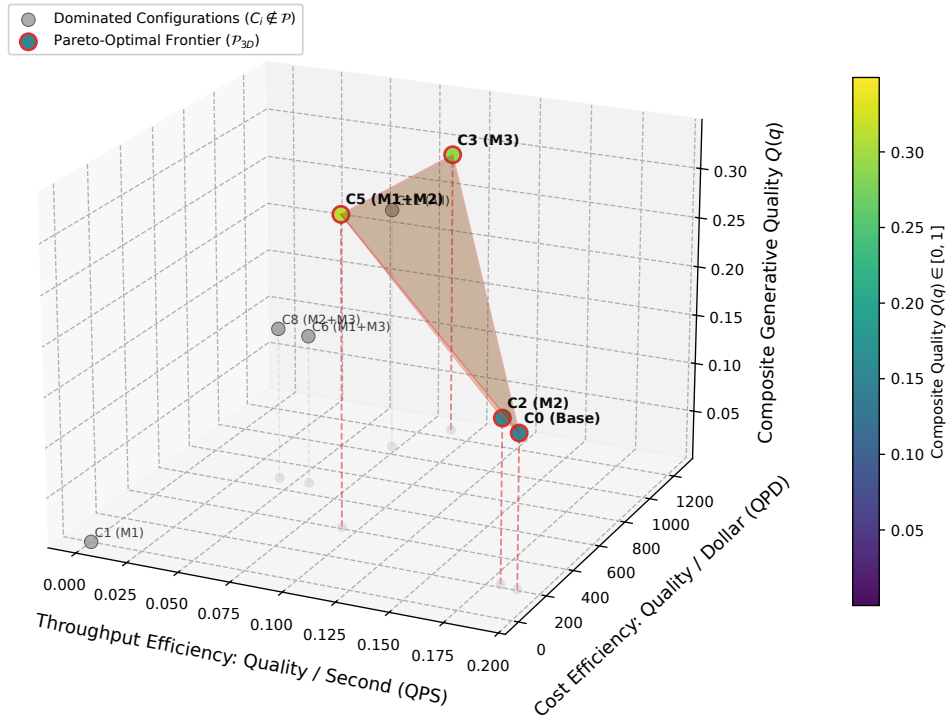


Figure 5: Empirical 3D Pareto frontier ($\mathcal{P}_{3D}$) across RAG pipeline configurations in tri-objective space: Throughput Efficiency (Quality per Second, QPS), Cost Efficiency (Quality per Dollar, QPD), and Composite Generative Quality $Q(q)$ **[SYNTHETIC MOCK DATA — FOR STRUCTURAL VALIDATION ONLY]**. Dashed vertical drop-lines connect coordinates to floor shadow projections; the shaded amber mesh outlines the non-dominated Pareto hull.

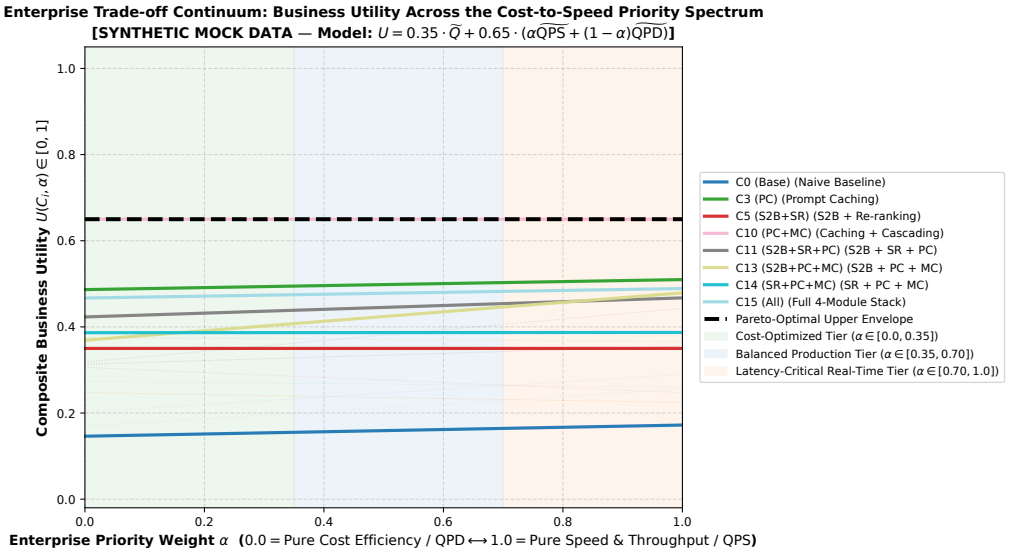


Figure 6: Enterprise trade-off continuum: Composite business utility $U(C_i, \alpha)$ as organizational priority shifts from pure cost efficiency ($\alpha = 0.0$) to pure throughput/speed ($\alpha = 1.0$) across the three Enterprise Production Archetypes **[SYNTHETIC MOCK DATA — FOR STRUCTURAL VALIDATION ONLY]**. The dashed black curve represents the upper Pareto envelope.

Table 8: Summary resolution of thesis research questions mapped to empirical findings.

RQ	Core Focus	Empirical Findings and Resolution
RQ1	**Latency Profile**	Two-stage re-ranking (M_2) and child-to-parent expansion (M_1) increase median latency, while prompt caching (M_3) reduces latency by bypassing prefill compute and accelerating TTFT. Model cascading (M_4) reduces median latency on simple queries but expands tail latency on complex multi-hop tasks.
RQ2	**Operational Cost**	Prompt caching (M_3) and model cascading (M_4) yield substantial operational cost reductions. Pairing caching with cascading (C_{10}) achieves maximum cost minimization. Token expansion in M_1 increases prompt token volume, but expenses are neutralized when paired with caching (C_6).
RQ3	**Generation Quality**	Small-to-Big retrieval (M_1) maximizes Context Recall, while Semantic Re-ranking (M_2) maximizes Context Precision. Their combination (C_5) achieves peak Composite Quality, significantly suppressing hallucinations. Caching maintains identical quality.
RQ4	**Combinations & Pareto**	Re-ranking permutes chunk prefixes, reducing downstream prompt cache hit rates (C_8). Stacking higher-order modules ($C_{11} - C_{15}$) exhibits diminishing quality returns while accumulating tail latency (P_{95}). Three distinct enterprise deployment archetypes define the Pareto frontier ($\mathcal{P}$).

6 Discussion

The idea of discussion is to bring up points that you noticed during research. As a chapter, this is the most freeform one. A good way to discuss the points is first to write up subtitles with clear statements and then expand on those. Discussion points can occasionally be also in a question format or refer to a larger context beyond the thesis to bring in ideas from other papers to consider. In case your work has limitations that you can observe, it can be a good idea to write a separate section considering those.

7 Conclusion

The idea of the conclusion is the opposite of the introduction. Here we literally bring the reader out of the topic, not in. A good way to write conclusion is to start by reminding the reader of your research question(s) and then state how your work addressed them. To finish your conclusion, consider open research questions (this can be a list for example).

References

[1] Stanford University Human-Centered Artificial Intelligence Institute. The Artificial Intelligence Index Report 2026. Technical report, Stanford University, April 2026. URL https:///hai.stanford.edu/ai-index/2026-ai-index-report.

[2] Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[3] Google Cloud. *Vertex AI Pricing: Generative AI and Foundation Models*, 2026. URL https://cloud.google.com/vertex-ai/generative-ai/pricing. Accessed September 2026.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[5] Shangyu Wu, Ying Xiong, Yufei Cui, Haolun Wu, Can Chen, Ye Yuan, Lianming Huang, Xue Liu, Tei-Wei Kuo, Nan Guan, and Chun Jason Xue. Retrieval-augmented generation for natural language processing: A survey. *arXiv preprint arXiv:2407.13193*, 2025.

[6] Siyun Zhao, Yuqing Yang, Zilong Wang, Zhiyuan He, Luna K. Qiu, and Lili Qiu. Retrieval augmented generation (rag) and beyond: A comprehensive survey on how to make your llms use external data more wisely. *arXiv preprint arXiv:2409.14924*, 2024.

[7] Broadcom Inc. Spring ai. https://spring.io/projects/spring-ai, 2026. Accessed: 2026-06-24.

[8] Elias Lumer, Matt Melich, Olivia Zino, Elena Kim, Sara Dieter, Pradeep Honaganahalli Basavaraju, Vamse Kumar Subbiah, James A. Burke, and Roberto Hernandez. Rethinking retrieval: From traditional retrieval augmented generation to agentic and non-vector reasoning systems in the financial domain for large language models. *arXiv preprint arXiv:2511.18177*, 2025.

[9] Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoeybi, and Bryan Catanzaro. Rankrag: Unifying context ranking with retrieval-augmented generation in llms. In *38th Conference on Neural Information Processing Systems (NeurIPS 2024)*, 2024.

[10] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of the 5th MLSys Conference*, 2024.

[11] American Bar Association. Bc tribunal confirms companies remain liable for information provided by ai chatbot. https://www.americanbar.org/groups/business_law/resources/business-law-today/2024-february/bc-tribunal-confirms-companies-remain-liable-information-provided-ai-chatbot/, February 2024. Accessed: 2026-06-24.

[12] Antti-Jussi Asunmaa. Large language models-based agents in software development. Bachelor's thesis, Aalto University, 2025. URL https://aaltodoc.aalto.fi/items/80839987-d1bc-46ac-8e2f-7f051e8b849e.

[13] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson Education, Inc., Upper Saddle River, New Jersey, third edition, 2010. ISBN 978-0-13-604259-4.

[14] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, et al. A survey of large language models. *Frontiers of Computer Science*, 20(12):2012627, 2026.

[15] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language Models are Unsupervised Multitask Learners. 2019.

[16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017*, pages 5998–6008, Long Beach, CA, USA, December 4-9 2017. Neural Information Processing Systems (NIPS).

[17] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

[18] Tristan Coignion, Clément Quinton, and Romain Rouvoy. A Performance Study of LLM-Generated Code on Leetcode. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, EASE 2024, page 79–89. ACM, June 2024. doi: 10.1145/3661167.3661221.

[19] Numa. Dhamani. *Introduction to Generative AI*. Manning Publications Co. LLC, New York, 1st ed. edition, 2024. ISBN 9781638354345.

[20] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12: 157–173, 2024.

[21] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM computing surveys*, 55(12):1–38, 2023.

[22] Gemini Team, Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. Technical report, Google DeepMind, 2024. URL https://arxiv.org/abs/2403.05530.

[23] Google DeepMind. Gemini 3 Flash Model Card. Model card, Google, December 2025. URL https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Flash-Model-Card.pdf.

[24] Bhuwan Dhingra, Jeremy R Cole, Julian Martin Eisenschlos, Daniel Gillick, Jacob Eisenstein, and William W Cohen. Time-aware language models as temporal knowledge bases. *Transactions of the Association for Computational Linguistics*, 10:257–273, 2022.

[25] Paulo Roberto de Moura Júnior, Jean Lelong, and Annabelle Blangero. Adaptive chunking: Optimizing chunking-method selection for rag. *arXiv preprint arXiv:2603.25333*, 2026.

[26] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, Haofen Wang, et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1):32, 2023.

[27] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE transactions on big data*, 7(3):535–547, 2019.

[28] Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. Retrieval augmentation reduces hallucination in conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 3784–3803, 2021.

[29] Bo Ni, Zheyuan Liu, Leyao Wang, Yongjia Lei, Yuying Zhao, Xueqi Cheng, Qingkai Zeng, Luna Dong, Yinglong Xia, Krishnaram Kenthapadi, et al. Towards trustworthy retrieval augmented generation for large language models: A survey. *arXiv preprint arXiv:2502.06872*, 2025.

[30] Claes Wohlin. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th international conference on evaluation and assessment in software engineering*, pages 1–10, 2014.

[31] Hao Yu, Aoran Gan, Kai Zhang, Shiwei Tong, Qi Liu, and Zhaofeng Liu. Evaluation of retrieval-augmented generation: A survey. In Wenwu Zhu, Hui Xiong, Xiuzhen Cheng, Lizhen Cui, Zhicheng Dou, Junyu Dong, Shanchen Pang, Li Wang, Lanju Kong, and Zhenxiang Chen, editors, *Big Data*, pages 102–120, Singapore, 2025. Springer Nature Singapore. ISBN 978-981-96-1024-2.

[32] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. Adaptive-rag: Learning to adapt retrieval-augmented large language models

through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7036–7050, 2024.

[33] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. In *International Conference on Learning Representations (ICLR 2025)*, 2025.

[34] Minghao Guo, Qingcheng Zeng, Xujiang Zhao, Yanchi Liu, Wenchao Yu, Mengnan Du, Haifeng Chen, and Wei Cheng. Deepsieve: Information sieving via LLM-as-a-knowledge-router. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 3054–3077, 2026.

[35] Jiarui Zhang, Xiangyu Liu, Yong Hu, Chaoyue Niu, Fan Wu, and Guihai Chen. Ragrouter: Learning to route queries to multiple retrieval-augmented language models. In *39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, 2025.

[36] Jaewan Park, Solbee Cho, and Jay-Yoon Lee. Stop-rag: Value-based retrieval control for iterative rag. In *39th Conference on Neural Information Processing Systems (NeurIPS 2025) Workshop: Multi-Turn Interactions in Large Language Models*, 2025.

[37] Zihang Li, Wenjun Liu, Yikun Zong, Jiawen Tao, Siying Dai, Songcheng Ren, Zirui Liu, Yuhang Wang, Yanbing Jiang, and Tong Yang. Bridge-rag: An abstract bridge tree-based retrieval augmented generation algorithm. *arXiv preprint arXiv:2603.26668*, 2026.

[38] Shengyuan Chen, Chuang Zhou, Zheng Yuan, Qinggang Zhang, Zeyang Cui, Hao Chen, Yilin Xiao, Jiannong Cao, and Xiao Huang. LogicRAG: Retrieval augmented generation with adaptive reasoning structures. In *The Fortieth AAAI Conference on Artificial Intelligence (AAAI-26)*, 2026.

[39] Minsang Kim, Cheoneum Park, and Seungjun Baek. Qpaug: Question and passage augmentation for open-domain question answering of llms. *arXiv preprint arXiv:2406.14277v2*, 2024.

[40] Mateusz Śmigielski, Michał Rajkowski, Mateusz Zbrocki, Michał Bernacki-Janson, Karol Kunicki, Julianna Godziszewska, Maciej Piasecki, and Konrad Wojtasik. Chunking methods on retrieval-augmented generation – effectiveness evaluation against computational cost and limitations. *arXiv preprint arXiv:2606.00881*, 2026.

[41] Gabriel de Souza P. Moreira, Ronay Ak, Benedikt Schifferer, Mengyao Xu, Radek Osmulski, and Even Oldridge. Enhancing q&a text retrieval with ranking models: Benchmarking, fine-tuning and deploying rerankers for rag. *arXiv preprint arXiv:2409.07691*, 2024.

[42] Zhiyuan Cheng, Longying Lai, Yue Liu, Kai Cheng, and Xiaoxi Qi. Enhancing financial report question-answering: A retrieval-augmented generation system with reranking analysis. *arXiv preprint arXiv:2603.16877*, 2026.

[43] Zihan Wang, Zihan Liang, Zhou Shao, Yufei Ma, Huangyu Dai, Ben Chen, Lingtao Mao, Chenyi Lei, Yuqing Ding, and Han Li. Infogain-rag: Boosting retrieval-augmented generation through document information gain-based reranking and filtering. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7190–7204. Association for Computational Linguistics, 2025.

[44] Zhepei Wei, Wei-Lin Chen, and Yu Meng. Instructrag: Instructing retrieval-augmented generation via self-synthesized rationales. In *Published as a conference paper at ICLR 2025*, 2025.

[45] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. Ragcache: Efficient knowledge caching for retrieval-augmented generation. *ACM Transactions on Computer Systems (TOCS)*, 44(1):2:1–2:27, 2025.

[46] Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. Turborag: Accelerating retrieval-augmented generation with precomputed kv caches for chunked text. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6588–6601. Association for Computational Linguistics, 2025.

[47] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the 20th European Conference on Computer Systems (EuroSys '25)*, pages 95–110. ACM, 2025.

[48] Jiahao Wang, Weiyu Xie, Mingxing Zhang, Boxing Zhang, Jianwei Dong, Yuening Zhu, Chen Lin, Jingqi Tang, Yaochen Han, Zhiyuan Ai, Xianglin Chen, Yongwei Wu, and Congfeng Jiang. From prefix cache to fusion rag cache: Accelerating llm inference in retrieval-augmented generation. *Proc. ACM Manag. Data*, 4(1):41:1–41:28, 2026.

[49] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, and Ali Farhadi. Matryoshka representation learning. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 30233–30249. Curran Associates, Inc., 2022. doi: 10.52202/068431-2192. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/c32319f4868da7613d78af9993100e42-Paper-Conference.pdf.

[50] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.

[51] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, 2024.

[52] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg, 2012. ISBN 978-3-642-29043-5. doi: 10.1007/978-3-642-29044-2.

[53] Google Cloud. *Vertex AI Search and Ranking API Pricing*, 2026. URL https://cloud.google.com/vertex-ai/generative-ai/pricing#ranking-api. Accessed September 2026.

[54] Anthropic. *Claude API Pricing and Prompt Caching Tariffs*, 2026. URL https://www.anthropic.com/pricing. Accessed September 2026.

A Enterprise Knowledge Corpus Extraction Methodology

This appendix is reserved for documenting the enterprise knowledge corpus extraction, deduplication, and preprocessing pipeline applied to the partner enterprise documentation corpus.

B Operational Cost Catalog and Cloud Tariffs

Table B1: Operational pricing catalog and cloud tariffs for generative, re-ranking, and embedding services (verified as of September 2026).

Model / Service	Pipeline Role	Prompt ($/1M)	Cached ($/1M)	Output ($/1M)	Auxiliary Unit Rate	Source
Gemini 3.8 Flash	Primary Workhorse (M_0, M_4)	$0.75	$0.075	$3.75	–	[3]
Gemini 3.5 Flash-Lite	Router Classifier / Fast Tier (M_4)	$0.075	$0.01875	$0.30	–	[3]
Gemini 3.1 Pro	Complex Multi-Hop Tier (M_4)	$1.25	$0.3125	$5.00	–	[3]
Vertex AI Ranking	Cross-Encoder Re-ranking (M_2)	–	–	–	$0.00010/doc	[53]
gemini-embedding-2	Dense Vectorization	$0.025	–	–	–	[3]
Claude Sonnet 5	Evaluation Judge (RAGAS)	$3.00	$0.30	$15.00	–	[54]